

Technical Report: Concurrent HTTP Web Server Implementation

1. Introduction

This project demonstrates the implementation of a functional, concurrent web server using the **C programming language** on a Linux-based environment (WSL). The server is designed to handle multiple simultaneous HTTP GET requests, serving static content such as HTML files.

2. System Architecture

The server follows a **Multi-threaded (Thread-per-request)** architecture. Unlike iterative servers, this model ensures high availability and responsiveness.

- **Socket Programming:** Utilized `socket()`, `bind()`, `listen()`, and `accept()` system calls to establish a reliable TCP connection on port **8080**.
 - **Concurrency Model:** Leveraged the **POSIX Threads (pthread)** library. For every incoming connection, a dedicated worker thread is spawned using `pthread_create()`.
 - **Resource Management:** Implemented `pthread_detach()` to ensure that thread resources are automatically reclaimed by the OS upon completion, preventing memory leaks and "zombie threads."
-

3. Key Features & Extensions (Bonus)

To enhance the robustness of the server, the following features were implemented:

- **Request Parsing:** The server dynamically parses the HTTP request header to extract the requested file path using `sscanf`.
 - **Security (Directory Traversal Protection):** Implemented a security layer to prevent unauthorized access. The server scans for `..` in the request path to block attempts to access sensitive files outside the root directory.
 - **Automated Logging:** Integrated a logging system that records every client interaction (Method, Path, and Status) into a persistent `server.log` file for auditing purposes.
 - **Robust File I/O:** Used binary mode (`rb`) for file operations, allowing the server to handle various MIME types, including text and images.
-

4. How to Run

1. **Compilation:** `gcc server.c -o server -lpthread`
 2. **Execution:** `./server`
 3. **Testing:** Access the server via browser: `http://localhost:8080/index.html`
-

5. Conclusion

This implementation provides a deep dive into **Operating Systems** concepts, specifically concurrency, synchronization, and file system management. The server successfully balances performance through multi-threading and security through path validation.
